

Datenstrukturen und Entwurfsmethoden

Ahmed Chtioui, Asma Mejbri, Mohamed Islem Slimane



12.11.2025

Warum Datenstrukturen & Algorithmen wichtig?

- Grundlegende Bausteine jeder Software
- Schnellere und strukturiertere Programme
- Effiziente Verarbeitung großer Datenmengen
- ermöglichen die Automatisierung von Aufgaben und Prozessen
- Häufig in Interviews vorkommend

Überblick über die Datenstrukturen und Entwurfsmethoden

Lineare Datenstrukturen

- Listen
- Arrays
- Stacks

Nicht-lineare Strukturen

- Hashing
- Heaps

Algorithmenentwurfsmethoden

- Greedy
- Divide & Conquer
- Brute-Force
- Backtracking

Listen

Definition

Reihe von Elementen, die durch Verweise miteinander verbunden sind.

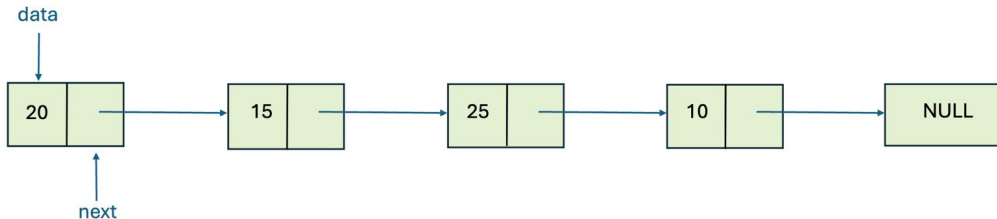
Merkmale

- Dynamische Speicherverwaltung
- Zugriff erfolgt über Zeiger oder Referenzen
- Kein direkter Zugriff über Index

1. Verkettete Liste

Definition

Eine Liste, bei der jeder Knoten auf den nächsten zeigt.



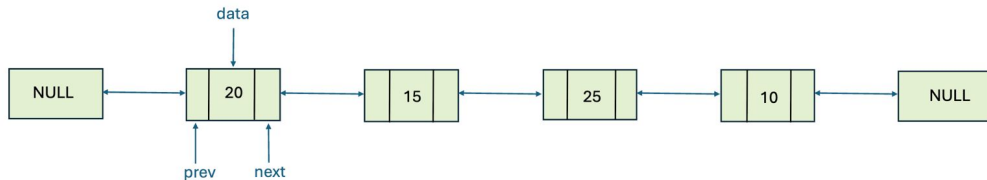
Merkmale:

- Lineare Verbindung der Elemente
- Bewegung nur in eine Richtung

2. Doppelt verkettete Liste

Definition

Eine Liste, bei der jeder Knoten auf den vorherigen und den nächsten zeigt.



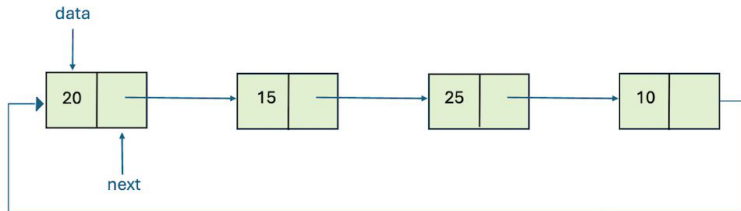
Merkmale:

- Bewegung in beide Richtungen möglich (praktischer)
- Mehr Speicherbedarf wegen zwei Zeigern

3. Zirkuläre Liste

Definition

Eine Liste, bei der der letzte Knoten wieder auf den ersten zeigt.



Merkmale:

- Ermöglicht endloses Durchlaufen der Liste
- Praktisch für zyklische Warteschlangen

Vergleich der Listenarten

Laufzeiten der wichtigsten Operationen

Operation	Verkettete Liste	Doppelt verkettete Liste	Zirkuläre Liste
Einfügen am Anfang	$O(1)$	$O(1)$	$O(1)$
Einfügen am Ende	$O(n)$	$O(1)$	$O(1)$
Einfügen an Position	$O(n)$	$O(n)$	$O(n)$
Suche	$O(n)$	$O(n)$	$O(n)$
Löschen am Anfang	$O(1)$	$O(1)$	$O(1)$
Löschen am Ende	$O(n)$	$O(1)$	$O(1)$
Löschen an Position	$O(n)$	$O(n)$	$O(n)$

Arrays

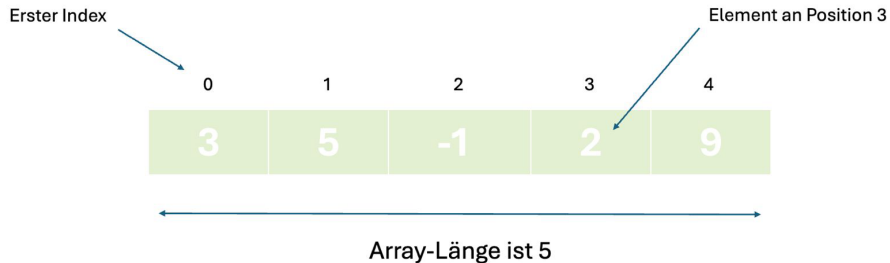
Definition

Sammlung von Elementen gleichen Typs in zusammenhängenden Speicherbereichen.

Merkmale

- Lineare Anordnung
- Homogener Datentyp
- Einfache Iteration
- Direkter Zugriff

Wie funktioniert ein Array?



Beispiel eines Arrays

- Jedes Feld repräsentiert ein Element im Speicher
- Zugriff über Index (z. B. `array[2]`)

Arraytypen

Zwei Haupttypen von Arrays:

1. Statische Arrays

- Feste Größe
- Einfache Speicherverwaltung
- Unflexibel bei unbekannter Datenmenge

2. Dynamische Arrays

- Größe zur Laufzeit anpassbar
- Flexibler Speichergebrauch
- Manuelle Freigabe erforderlich

Laufzeit von Arrays

Laufzeiten der wichtigsten Operationen

Operation	Statisches Array	Dynamisches Array
Zugriff auf Element	$O(1)$	$O(1)$
Einfügen am Ende	$O(n)$	$O(1)$
Einfügen an Position	$O(n)$	$O(n)$
Löschen am Ende	$O(n)$	$O(1)$
Löschen an Position	$O(n)$	$O(n)$
Suche	$O(n)$	$O(n)$

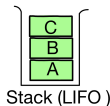
Stacks

Definition

Datenstruktur, bei der Elemente oben hinzugefügt und entfernt werden.(LIFO: last In, First Out).

Merkmale:

- Zugriff nur auf das oberste Element möglich
- Wichtige Operationen: push, pop
- Beispiel: Bücherstapel : das oberste Buch wird zuerst entfernt



Operationen

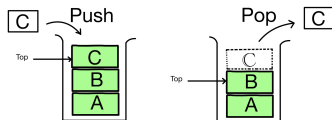
Hauptoperationen

- **push(x)** : Fügt ein neues Element x oben auf den Stack.
- **pop()** : Entfernt das oberste Element und gibt es zurück.

Beispiel

Angenommen, der Stack enthält [A, B] (B ist oben):

- $\text{push}(C) \Rightarrow [A, B, C]$
- $\text{pop}() \Rightarrow [A, B]$



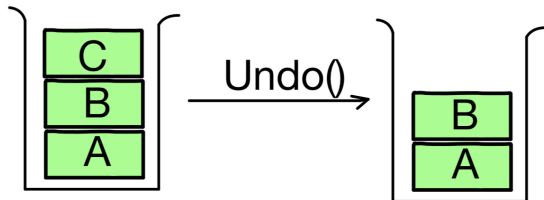
Anwendung von Stacks

Beispiel: Rückgängig-Funktion (Undo)

Ein Stack kann genutzt werden, um Aktionen rückgängig zu machen.

Ablauf:

- Aktionen werden nacheinander auf den Stack gelegt (push)
- Beim Rückgängig-Machen wird die letzte Aktion entfernt (pop)



Hashing

Definition

Ein Verfahren, bei dem eine Eingabe beliebiger Länge mit Hilfe einer bestimmten mathematischen Funktion in einen Wert fester Länge (**Hashwert**) umgewandelt wird.

Beispiel:

- **Hash-Funktion:**

$$f(x) = (\text{Summe der Buchstabenpositionen im Alphabet}) \bmod 7$$

- **Berechnung:**

$$f(\text{„Mike“}) = (M + i + k + e) = (13 + 9 + 11 + 5) = 38$$

- **Ergebnis:**

$$38 \bmod 7 = 3 \Rightarrow f(\text{„Mike“}) = 3$$

Hashfunktionen in der Praxis

Anwendungen:

- Datenbanken
- Passwortspeicherung
- Kryptographie & Blockchain (z. B. SHA-256)

Vorteile:

- + Konstante Zugriffszeit: $O(1)$
- + Effizient bei großen Datenmengen

Nachteile:

- Kein sortierter Zugriff
- Abhängig von der Hashfunktion

Eigenschaften von Hashfunktionen

- **Deterministisch:** Gleicher Input ergibt denselben Hashwert

Beispiel: $\text{Hash}(\text{"Mike"}) = 3$, $\text{Hash}(\text{"Mike"}) = 3$

- **Kollisionsfrei:** Unterschiedliche Eingaben haben verschiedene Hashwerte

Beispiel: $\text{Hash}(\text{"Mike"}) = 11$, $\text{Hash}(\text{"John"}) = 11$ ❌

- **Avalanche-Effekt:** Kleine Änderungen im Input erzeugen stark unterschiedliche Hashwerte

Beispiel: $\text{Hash}(\text{"Mike"}) = 1230$, $\text{Hash}(\text{"Mika"}) = 246$

- **Irreversibel:** Aus dem Output kann der ursprüngliche Input nicht gefunden werden

Beispiel: $\text{Hash}(\text{unbekannt}) = 15$

Heap – Definition & Eigenschaften

Definition

eine baumbasierte Datenstruktur, die die **Heap-Eigenschaft** erfüllt:

- **Max-Heap:** Jeder Elternknoten ist größer oder gleich seinen Kindern
- **Min-Heap:** Jeder Elternknoten ist kleiner oder gleich seinen Kindern

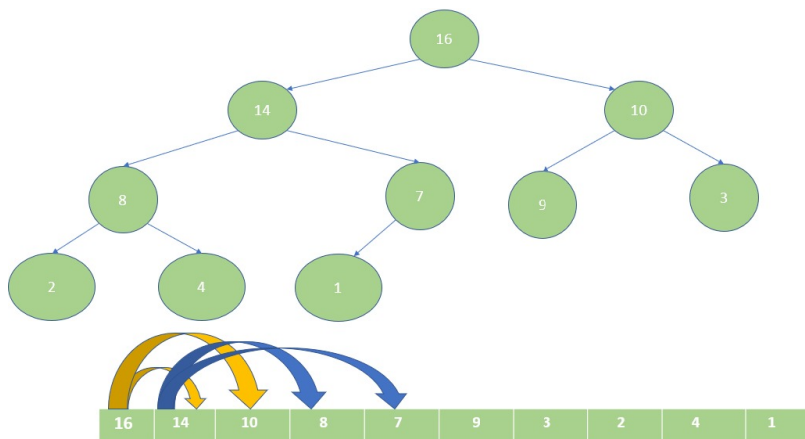
Eigenschaften:

- ein vollständiger binärer Baum
- typischerweise in einem Array gespeichert:

$$\text{left}(i) = 2i, \quad \text{right}(i) = 2i + 1, \quad \text{parent}(i) = \left\lfloor \frac{i}{2} \right\rfloor$$

z. B. $i = 1$ ist die Wurzel, $i = 2$ und $i = 3$ sind ihre Kinder usw.

Beispiel: Max-Heap



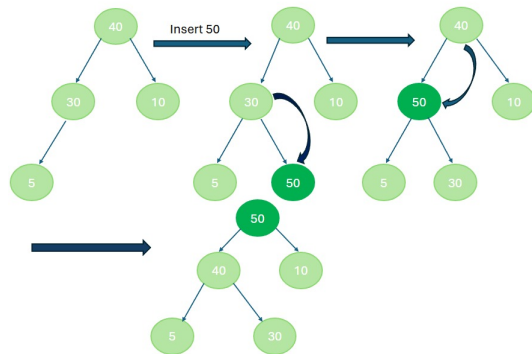
Wichtige Operationen

Operation	Beschreibung	Laufzeit
insert(heap,x)	Neues Element einfügen dann Ordnung durch heapifyUp wiederherstellen.	$\mathcal{O}(\log n)$
extract(heap)	Wurzel entfernen dann Ordnung durch heapifyDown	$\mathcal{O}(\log n)$
peek(heap)	Wurzel ausgeben ohne sie zu entfernen.	$\mathcal{O}(1)$
heapifyUp(heap,i)	Nach insert : Element steigt nach oben, solange größer(Max-Heap) / kleiner (Min-Heap) als Elternknoten.	$\mathcal{O}(\log n)$
heapifyDown(heap,i)	Nach extract : Wurzel sinkt nach unten, bis größer(Max-Heap) / kleiner(Min-Heap) als beide Kinder.	$\mathcal{O}(\log n)$

Beispiel: Insert beim Max-Heap

Schritte:

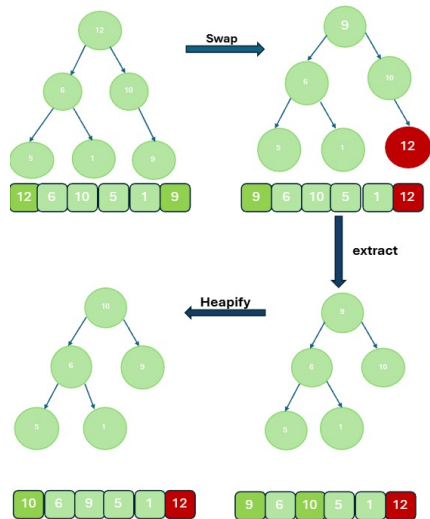
- Füge 50 am Ende ein $\rightarrow [40, 30, 10, 5, 50]$
- Vergleiche mit Eltern (30): $50 > 30 \rightarrow$ tauschen
- Vergleiche mit nächstem Eltern (40): $50 > 40 \rightarrow$ tauschen



Beispiel: Extract beim Max-Heap

Schritte:

- Entferne Wurzel (12) → letztes Element (9) nach oben
- Vergleiche mit Kindern (6, 10): 10 ist größer → tauschen
- Vergleiche erneut: $9 > 5$ → Heap korrekt



Anwendungen von Heaps

Common Uses

- **Priority Queues:** Verwaltung von Aufgaben nach Priorität
- **Heapsort:** Sortieralgorithmus mit Laufzeit $O(n \log n)$
- **Dijkstra-Algorithmus:** Kürzeste Wege in Graphen
- **Prim-Algorithmus:** Minimaler Spannbaum
- **Scheduling-Systeme:** Prozesse mit Priorität

spätere Vorträge dazu...

Greedy

Definition

Ein Verfahren, bei dem in jedem Schritt die lokal beste Entscheidung gewählt wird, um am Ende die möglichst optimale Lösung zu erreichen.

Merkmale:

- Wählt stets die beste sofortige Option
- Keine Rückverfolgung (Backtracking)
- Einfach und schnell, aber nicht immer optimal

Funktionseise

Allgemeiner Ablauf

- ① Initialisiere Lösung (leer)
- ② Wiederhole, solange Elemente verfügbar sind:
 - ▶ Wähle das **beste verbleibende Element**
 - ▶ Füge es der Lösung hinzu
 - ▶ Überprüfe, ob die Lösung gültig bleibt
- ③ Gib die gebildete Lösung zurück

Beispiel: Münzwechselproblem

Problem

Gegeben: Münzen mit den Werten 50, 20, 10, 5, 2, 1

Gesucht: Minimale Anzahl Münzen für einen Betrag.

Greedy-Schritte für Betrag = 76:

- ➊ Starte mit Restbetrag = 76
- ➋ Wähle größte mögliche Münze:
 - ▶ $76 - 50 = 26$
 - ▶ $26 - 20 = 6$
 - ▶ $6 - 5 = 1$
 - ▶ $1 - 1 = 0$
- ➌ Lösung: **4 Münzen (50, 20, 5, 1).**

Ergebnis: Minimal = 4 Münzen für Betrag 76

Divide & Conquer

Definition

Ein Ansatz, bei dem ein großes Problem in kleinere Teilprobleme geteilt wird, diese einzeln gelöst und schließlich zu einer Gesamtlösung kombiniert werden.

Funktionsweise:

- **Divide:** Das Problem wird in kleinere Teilprobleme zerlegt.
- **Conquer:** Die Teilprobleme werden rekursiv gelöst und kombiniert.

Vorteile:

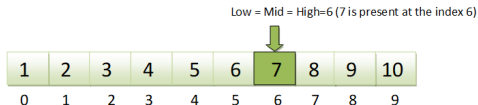
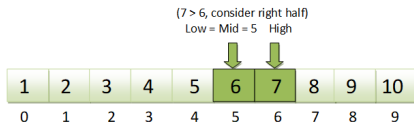
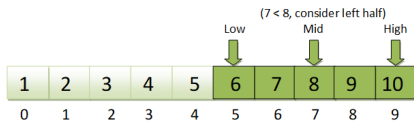
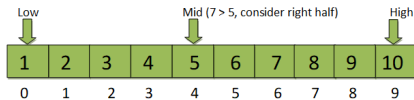
- + Gute Laufzeit bei großen Problemen
- + Klare, strukturierte Vorgehensweise durch Rekursion

Nachteile:

- Zusätzlicher Speicherbedarf durch Rekursion
- Rekursive Implementierung kann komplex sein

Beispiel: Binary search

Ziel: finde die Zahl 7



Brute Force

Definition

Ein Brute-Force-Algorithmus prüft **alle möglichen Lösungen**, bis die **richtige** gefunden ist.

Merkmale:

- Systematische Suche durch alle Möglichkeiten
- Garantiert eine korrekte Lösung (wenn vorhanden)
- Einfach zu verstehen und zu implementieren
- Sehr hohe Laufzeit für große Eingaben

Funktionsweise

Allgemeiner Ablauf

- 1 Erzeuge alle möglichen Kandidatenlösungen
- 2 Für jede Lösung:
 - ▶ Überprüfe, ob sie gültig ist
 - ▶ Bewerte ihre Qualität (z. B. Kosten, Länge, etc.)
- 3 Wähle die **beste gültige Lösung**

Beispiel :

Passwortsuche:

data	→	X
struktur	→	X
projekt	→	✓ Gefunden!

In diesem Fall ist „projekt“ das Passwort.

Brute Force testet alle Kombinationen, bis die richtige gefunden ist.

Backtracking

Definition

Eine Methode, bei der man eine Lösung schrittweise aufbaut, verschiedene Möglichkeiten ausprobiert und bei einem falschen Weg zurückgeht, um eine andere Option zu testen.

Merkmale:

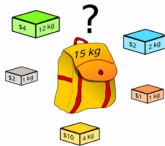
- Arbeitet nach dem „Versuch-und-Irrtum“-Prinzip (trial and error)
- Baut eine Teillösung Schritt für Schritt auf
- Wenn eine Regel verletzt wird → **Rückkehr zum vorherigen Zustand**
- Wird häufig mit **Rekursion** implementiert

Funktionsweise

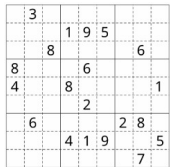
Pseudocode

```
Funktion FindeLösung(Stufe, Vektor):  
    solange es noch Möglichkeiten gibt:  
        wähle eine neue Möglichkeit  
        falls gültig (z.B. keine Regel verletzt wird):  
            füge sie zum Vektor hinzu #Teil-Lösung erweitern  
            falls vollständig:  
                return True    #Lösung gefunden  
            sonst:  
                FindeLösung(Stufe+1, Vektor)  
            mache letzte Wahl rückgängig #Backtracking  
    return False    #keine Lösung
```

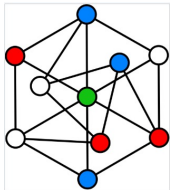
Beispiele:



Rucksackproblem: Wähle Gegenstände mit maximalem Wert, aber unterhalb einer Gewichtsgrenze.



Sudoku: Zahlen nach Regeln in ein Raster einfügen.



Färbeproblem: Knoten so einfärben, dass Nachbarknoten unterschiedliche Farben haben.

Laufzeit

Allgemein

Bei **Backtracking** werden viele Möglichkeiten ausprobiert, bis eine gültige Lösung gefunden wird. Das kann problematisch sein, wenn es **viele Optionen** gibt.

Laufzeit im schlechtesten Fall:

$$O(z^N)$$

- z = Anzahl der **Möglichkeiten pro Schritt**
- N = Anzahl der **benötigten Schritte**

Beispiel: Wenn es 7 Ebenen gibt und pro Stufe 4 Möglichkeiten bestehen:

$$4^7 = 16,384 \text{ mögliche Versuche}$$

→ **Fazit:** Die Laufzeit kann **exponentiell wachsen!**

Feedback

Bewertung des Backtracking-Algorithmus

- **Zuverlässig:** Findet immer eine Lösung, wenn eine existiert.
- **Nicht effizient:** sehr langsam, wenn es viele Möglichkeiten gibt.
- **Praktisch:** Gut geeignet für kleine Probleme

Fazit: Backtracking ist sicher, aber nicht ideal , wenn man lieber eine garantierte Lösung als maximale Effizienz möchte.

Problem

Aufgabenstellung

Gegeben ist ein unsortiertes Array mit $n > 5$ Zahlen.

Ziel: Sortiere das Array in aufsteigender Reihenfolge.

Herausforderung:

- Das Array ist zu groß, um jedes Element manuell zu vergleichen
- Direkte Sortierverfahren wie Bubble Sort sind ineffizient bei größeren Arrays
- Man möchte die Anzahl der Vergleiche und Verschiebungen minimieren

Frage an den Leser: Wie könnte man das Problem effizient lösen, indem man das Array in kleinere Teilprobleme zerlegt und anschließend die Teillösungen kombiniert?

Lösung

Pseudocode

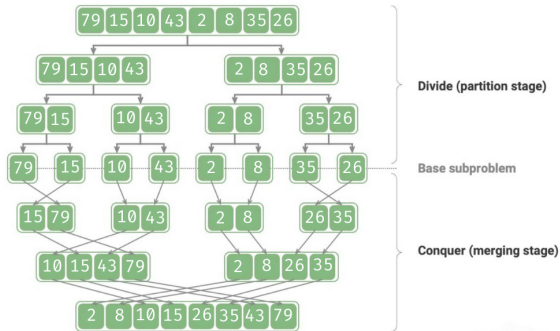
```
Funktion DivideAndConquerSort(A):  
    falls Länge(A) <= 1:  
        return A # Array ist bereits sortiert  
    # 1. Teile das Array in zwei Hälften  
    mid = Länge(A) // 2  
    left  = A[0 : mid]  
    right = A[mid : end]  
    # 2. Sortiere beide Hälften rekursiv  
    leftSorted  = DivideAndConquerSort(left)  
    rightSorted = DivideAndConquerSort(right)  
    # 3. Führe die sortierten Hälften zusammen  
    return merge(leftSorted, rightSorted)
```

Beispiel:

Konkretes Array

Für das folgende Array soll die Sortierung demonstriert werden:

array = [79, 15, 10, 43, 2, 8, 35, 26]



Zusammenfassung

- Verstehen des Problemtyps
- Analyse von Zeit- und Speicherkomplexität
- Analyse der positiven und negativen Eigenschaften
- Beachten der Datenänderung und -größe

⇒ **Die zur Aufgabe passende Datenstruktur und der richtige Algorithmus sind für eine effiziente Problemlösung entscheidend.**

Fragen?

Vielen Dank für Ihre Aufmerksamkeit!